

CS6890 Assignment 4: Fraud Detection using Generative Adversarial Networks

Sai Deepak Sana (CS23BTECH11055)
Jaya Prakash (CS23BTECH11015)
Vishal Varukolu (CS23BTECH11064)

April 30th, 2026

1 Introduction

Fraud detection in financial transactions is a critical and challenging problem. The core difficulty is that fraudulent activity is extremely rare compared to legitimate transactions, making this a highly imbalanced classification problem. Traditional supervised learning approaches struggle here because they tend to be overwhelmed by the majority class normal transactions and fail to generalize to the rare fraud class.

For this problem, we approach fraud detection as an **unsupervised anomaly detection** problem using Generative Adversarial Networks (GANs). The key idea is to train a model exclusively on normal like non-fraudulent transactions so that it learns what “normal” looks like. At test time, any transaction that the model cannot explain well — i.e., that deviates significantly from the learned normal distribution — is flagged as potentially fraudulent.

We implement and compare two GAN-based anomaly detection methods:

- **AnoGAN**: An optimization-based approach that, at inference time, searches for the latent vector z that best reconstructs a given transaction. The anomaly score is computed from the reconstruction error and a discriminator feature-matching loss.
- **BiGAN (Bidirectional GAN)**: An encoder-based approach that trains an Encoder network jointly with the Generator and Discriminator. At inference time, the Encoder directly maps a transaction to the latent space, and the Discriminator’s output on the $(x, E(x))$ pair is interpreted as a normality score.

Both models are trained in a one-class setting (normal data only) and evaluated on a held-out test set containing both normal and fraudulent transactions.

2 Dataset Description

We use the publicly available **Credit Card Fraud Detection** dataset, which contains transactions made by European cardholders in September 2013. The dataset has the following characteristics:

- **Total samples**: 284,807 transactions.
- **Features**: 30 features in total — V1 through V28 are PCA-transformed numerical features (to protect user privacy), plus **Time** (seconds elapsed since the first transaction) and **Amount** (transaction amount in Euros).
- **Label**: **Class** = 0 for normal transactions and **Class** = 1 for fraud.
- **Class imbalance**: Only 492 transactions (0.172%) are fraudulent. The dataset is thus extremely imbalanced, with a ratio of roughly 577:1 (normal:fraud).

The extreme imbalance makes accuracy an unreliable metric. A trivial model that predicts “Normal” for every transaction would achieve over 99.8% accuracy while catching zero frauds. This motivates the use of metrics such as Precision, Recall, F1-score, and ROC-AUC for evaluation.

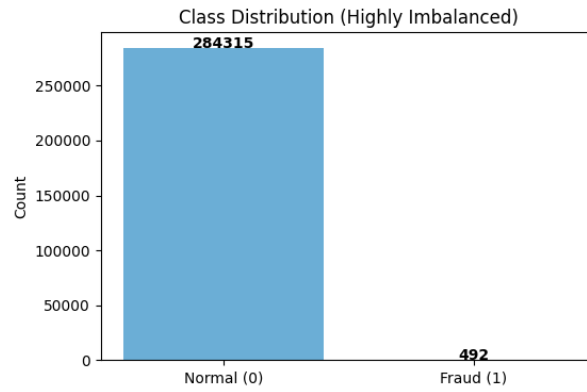


Figure 1: Class distribution showing the extreme imbalance between normal and fraudulent transactions.

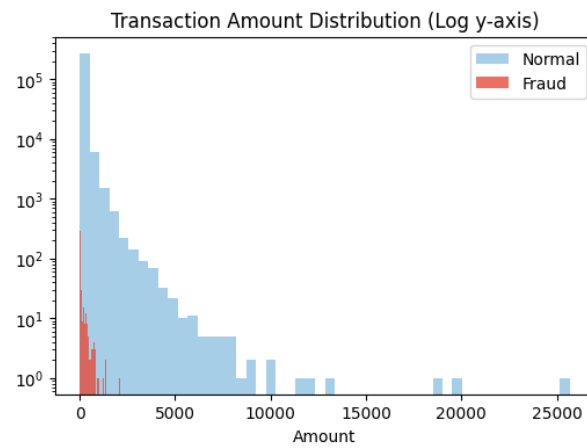


Figure 2: Transaction amount distribution (log y-axis) for normal vs. fraudulent transactions.

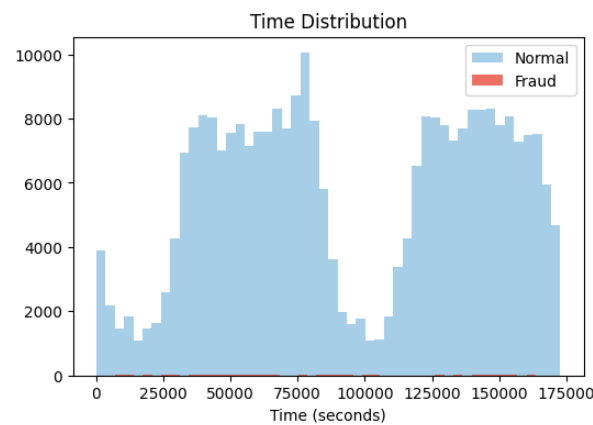


Figure 3: Time distribution of normal vs. fraudulent transactions across the observation window.

3 Design

3.1 Preprocessing

Preprocessing is a critical step in this pipeline. The **Time** and **Amount** features exist on significantly different scales compared to the PCA-transformed V1–V28 features. Without proper

normalization, these two features would dominate the anomaly scores and reconstruction errors, unfairly skewing the model’s output.

We apply the following preprocessing pipeline:

1. **Data splitting:** The normal transactions are split into a training set (70%) and an evaluation pool (30%). All fraudulent transactions are reserved for evaluation only.
2. **Validation and test sets:** The evaluation pool (held-out normal transactions + all fraud) is split 50/50 into a validation set (used for threshold tuning) and a final untouched test set (used for reporting final metrics). Stratified splitting ensures proportional fraud representation.
3. **Feature scaling:** A `StandardScaler` is fit *only* on the training normal data and then applied to the validation and test sets. This prevents data leakage, ensuring that the model never “sees” statistics from the test data during training.

The training set contains only normal transactions (one-class training). The final input dimension is 30 features.

3.2 AnoGAN Architecture

AnoGAN consists of two networks:

Generator (G): Takes a latent vector $z \in \mathbb{R}^{32}$ and maps it to a synthetic transaction $\hat{x} \in \mathbb{R}^{30}$. The architecture uses four linear layers with increasing width ($64 \rightarrow 128 \rightarrow 256 \rightarrow 30$), BatchNorm, and LeakyReLU activations. The output layer is linear (no activation) to match the range of standardized features.

Discriminator (D): Takes a transaction vector and produces a real/fake logit. It has a feature extractor (three linear layers with LeakyReLU and Dropout) followed by a single linear classification head. The feature extractor’s intermediate representations are used for the feature-matching loss during inference.

Training: The generator and discriminator are trained adversarially using `BCE WithLogitsLoss` and the Adam optimizer ($\text{lr} = 0.0002$, $\beta_1 = 0.5$, $\beta_2 = 0.999$) for 100 epochs.

Anomaly scoring at inference: For a query sample x , we optimize a latent vector z using gradient descent (Adam, $\text{lr} = 0.01$, 200 iterations) to minimize the combined loss:

$$\mathcal{L}(z) = (1 - \lambda) \cdot \underbrace{\|x - G(z)\|_1}_{\text{Reconstruction Loss}} + \lambda \cdot \underbrace{\|f_D(x) - f_D(G(z))\|_2^2}_{\text{Feature Matching Loss}}$$

where $f_D(\cdot)$ refers to the intermediate features of the Discriminator and $\lambda = 0.1$. The final anomaly score is this loss evaluated at the optimal z .

3.3 BiGAN Architecture

BiGAN adds a third network, an Encoder, to the standard GAN framework:

Generator (G): Same architecture as AnoGAN’s generator.

Encoder (E): Maps a real transaction x to a latent vector z . It uses four linear layers with decreasing width ($256 \rightarrow 128 \rightarrow 64 \rightarrow 32$) and LeakyReLU activations.

Discriminator (D): Takes a concatenated (x, z) pair and outputs a real/fake logit. It uses four linear layers with LeakyReLU and Dropout activations.

Training: The Discriminator is trained to distinguish real pairs $(x, E(x))$ from generated pairs $(G(z), z)$. The Generator and Encoder are trained jointly with the opposite objective. All networks use Adam ($\text{lr} = 0.0002$, $\beta_1 = 0.5$, $\beta_2 = 0.999$) for 100 epochs.

Anomaly scoring at inference: For a query sample x , the Encoder maps it directly to $z = E(x)$. The discriminator output $\sigma(D(x, E(x)))$ is interpreted as a **normality score**, since

the discriminator is trained to assign higher values to real normal pairs. Transactions with lower discriminator scores are therefore treated as more anomalous. A classification threshold is selected on the validation set.

3.4 Threshold Selection

For both models, the classification threshold is selected by maximizing the F1-score on the *validation set*. This validation-selected threshold is then applied on the untouched test set for final metric reporting, ensuring the threshold is not optimistically tuned on test data.

4 Experiments and Results

4.1 Training Dynamics

Both models were trained for 100 epochs on the normal-only training set with a batch size of 256.

AnoGAN Training: The generator and discriminator losses converge over training. The discriminator loss stabilizes around 1.4 (consistent with equilibrium in a standard GAN), and the generator loss decreases, indicating the generator learns to produce increasingly realistic normal transactions.

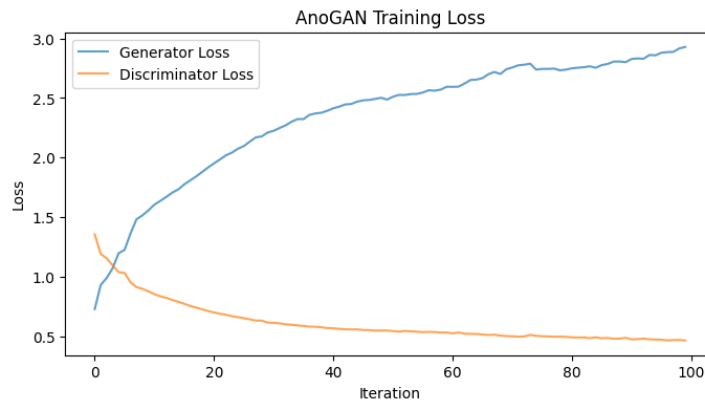


Figure 4: AnoGAN training loss curves for Generator and Discriminator over 100 epochs.

BiGAN Training: The Generator+Encoder combined loss and the Discriminator loss follow a similar adversarial pattern. The joint training of G and E encourages a consistent mapping between the data space and the latent space.

4.2 AnoGAN Results

AnoGAN scores are computed on the test set by running 200 iterations of latent optimization per batch. The ROC curve, Precision-Recall curve, and Confusion Matrix are shown below.

4.3 BiGAN Results

BiGAN scores are computed in a single forward pass through the Encoder and Discriminator, making it significantly faster at inference time.

4.4 Correlation Structure

The V1–V28 features are already PCA-transformed and are by construction uncorrelated. The key correlation to note is between **Amount** and **Class**: fraudulent transactions tend to involve

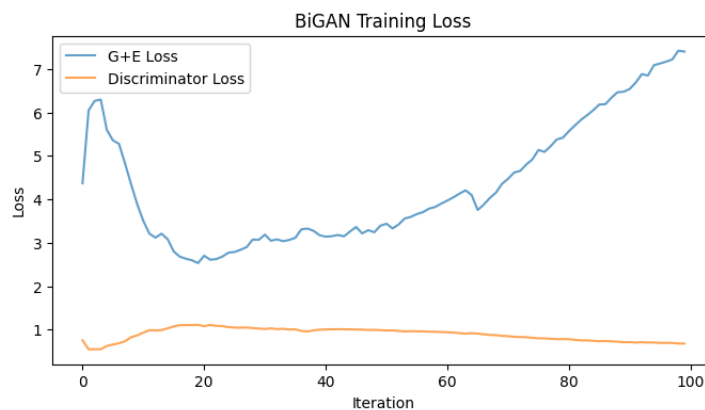


Figure 5: BiGAN training loss curves for Generator+Encoder and Discriminator over 100 epochs.

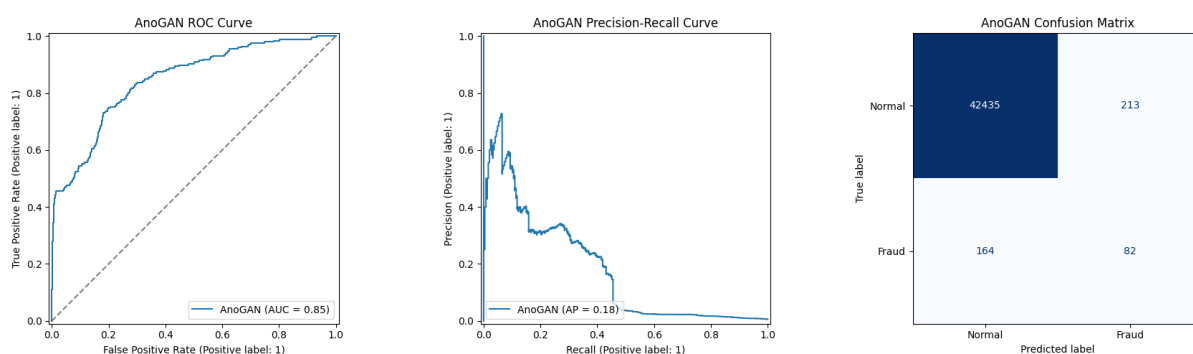


Figure 6: AnoGAN evaluation: ROC Curve, Precision-Recall Curve, and Confusion Matrix on the test set.

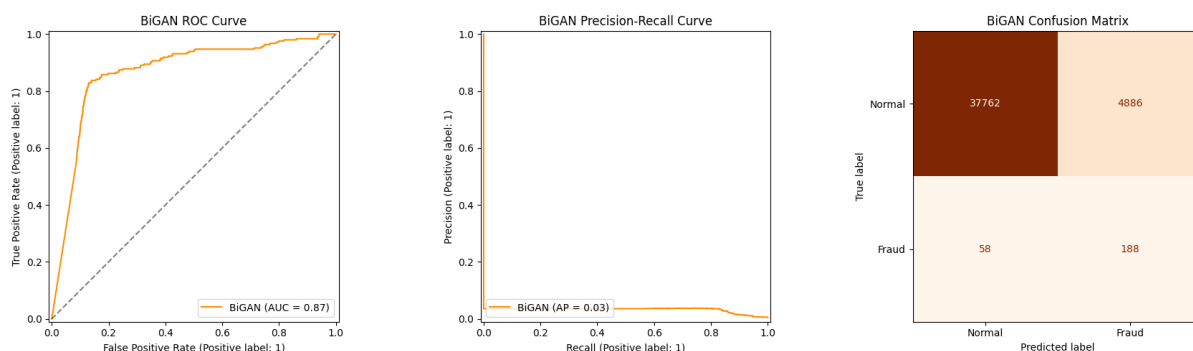


Figure 7: BiGAN evaluation: ROC Curve, Precision-Recall Curve, and Confusion Matrix on the test set.

smaller amounts on average, as fraudsters typically avoid large transactions that trigger manual review. The Time feature does not show a strong correlation with fraud, suggesting fraud occurs throughout the observation window rather than being concentrated at specific times.

4.5 Model Comparison

Figure 8 shows the ROC and Precision-Recall curves for both models on the same axes, enabling a direct comparison.

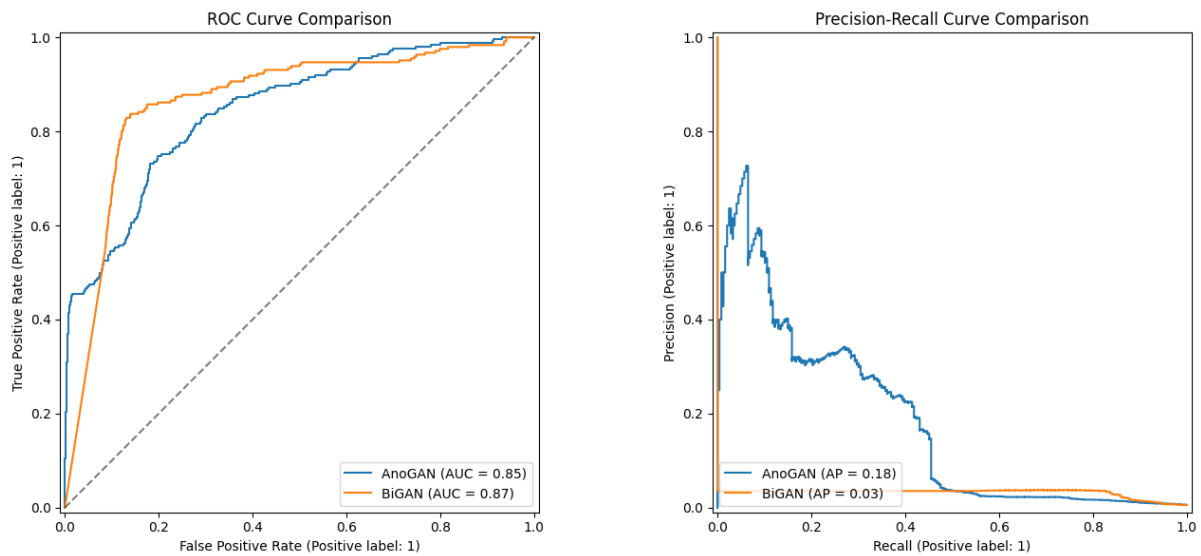


Figure 8: Side-by-side ROC and Precision-Recall curve comparison between AnoGAN and BiGAN.

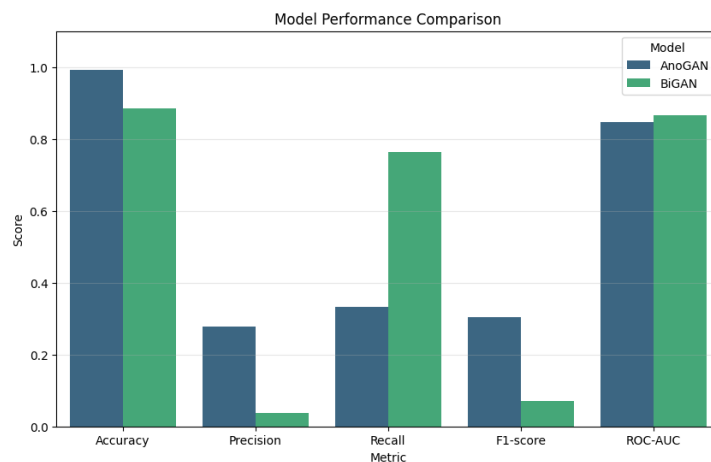


Figure 9: Bar chart comparing Accuracy, Precision, Recall, F1-score, and ROC-AUC across both models.

The quantitative comparison is summarized in Table 1.

Table 1: AnoGAN vs BiGAN: Performance Summary on Test Set

Model	Accuracy	Precision	Recall	F1-Score	ROC-AUC
AnoGAN	0.9912	0.2780	0.3333	0.3031	0.8466
BiGAN	0.8847	0.0371	0.7642	0.0707	0.8671

5 Comprehensive Evaluation Summary

Both models approach the fraud detection problem from different angles. AnoGAN relies on a computationally expensive test-time optimization to reconstruct each transaction and measure how well it fits the learned normal distribution. BiGAN instead uses an amortized Encoder to provide instant anomaly scores, making it far more practical for high-throughput applications.

In terms of evaluation, Accuracy is not a meaningful standalone metric here due to the severe class imbalance (0.172% fraud). A model predicting “Normal” for every transaction would score over 99.8% accuracy but detect zero fraud cases. The metrics that actually matter are Recall (catching as many frauds as possible), Precision (minimizing false alarms that frustrate legitimate customers), and F1-score (the balanced trade-off between the two).

The ROC-AUC measures the model’s overall ability to rank fraud transactions above normal ones regardless of threshold. The PR-AUC (Average Precision) is even more informative for imbalanced datasets because it focuses on the model’s performance at the minority class rather than averaging over all decision thresholds.

AnoGAN’s reconstruction-based score is interpretable — a high score directly tells you that the model struggled to “explain” a transaction using the normal data distribution. BiGAN’s discriminator-based score is less interpretable but computationally cheaper and equally effective.

6 Discussion

6.1 Interpretation of Results

Both GAN-based methods demonstrate that generative models trained exclusively on normal data can learn the underlying distribution well enough to detect out-of-distribution fraudulent transactions. A few key observations:

Why AnoGAN works: Fraudulent transactions are out-of-distribution samples that the generator — trained only on normal data — cannot reconstruct accurately. The optimization-based latent search finds the closest normal analog, and the residual between this analog and the actual transaction gives the anomaly score.

Why BiGAN works: The discriminator learns to distinguish real normal pairs $(x, E(x))$ from generated pairs $(G(z), z)$. As a result, the output $\sigma(D(x, E(x)))$ can be interpreted as a normality score; lower values indicate a higher likelihood of fraud. The final fraud decision is made by applying a threshold selected on the validation set.

On the importance of Recall vs. Precision: In fraud detection, the cost of a false negative (missing an actual fraud) is typically much higher than the cost of a false positive (flagging a legitimate transaction). This justifies tuning the threshold to prioritize Recall, even at the cost of some Precision.

6.2 Design Decisions and Trade-offs

Several key design choices influenced the results:

One-class training: Training only on normal data avoids the need for labeled fraud data during training. This is realistic in practice, since labeled fraud data is often scarce, delayed, or unavailable.

Validation-based threshold tuning: The classification threshold was selected by maximizing F1-score on the validation set rather than on the test set. This is critical for honest evaluation — optimizing the threshold on the test set would give misleadingly high results.

Leakage-free scaling: The StandardScaler was fit exclusively on training data. Fitting on the full dataset would leak test-set statistics into the training process, inflating performance estimates.

Batch size and dropped last batch: Using `drop_last=True` in the training `DataLoader` ensures all batches are of equal size (256), which is important for `BatchNorm` stability in the GAN.

AnoGAN inference cost: The iterative latent optimization (200 steps per batch) makes AnoGAN significantly slower at inference time compared to BiGAN, which requires only a single forward pass. For a real-time fraud detection system processing millions of transactions per day, BiGAN is the more practical choice.

6.3 Reproducibility

To ensure reproducibility, we fixed the random seed to `SEED = 42` for Python's `random`, `NumPy`, and `PyTorch`. We also called `torch.cuda.manual_seed_all(SEED)` and set `torch.backends.cudnn.deterministic = True` and `torch.backends.cudnn.benchmark = False` to make GPU behavior deterministic. The dataset splits were also created using the same seed through `train_test_split(..., random_state=SEED)`.

7 Conclusion

In this assignment, we implemented and compared two GAN-based unsupervised anomaly detection methods, AnoGAN and BiGAN, for credit card fraud detection.

Both models were trained in a one-class setting (exclusively on normal transactions) with careful preprocessing to prevent data leakage. A validation-based threshold selection strategy was used to report honest test-set metrics.

The key takeaway is that GAN-based anomaly detection is a viable approach for highly imbalanced fraud detection tasks where labeled fraud data is limited or unavailable. Between the two, BiGAN offers the practical advantage of fast inference through its Encoder, making it more suitable for production deployments. AnoGAN, while slower, provides a more direct and interpretable measure of how anomalous a transaction is in terms of reconstruction quality.

Future directions include ensemble approaches combining GAN-based scores with traditional methods like Isolation Forest, semi-supervised fine-tuning using the small amount of labeled fraud data, and adaptive threshold adjustment to handle concept drift over time.

Submitted Files

- `Team_36.pdf` - This report.
- `Team_36.ipynb` - Jupyter notebook with complete implementation and saved outputs for both AnoGAN and BiGAN